Falling Bricks Game

⚙️ Prerequisites

Make sure the following are installed before running the application:

1. Java 17
2. Maven

🚀 Run Application

Option 1: Using Maven

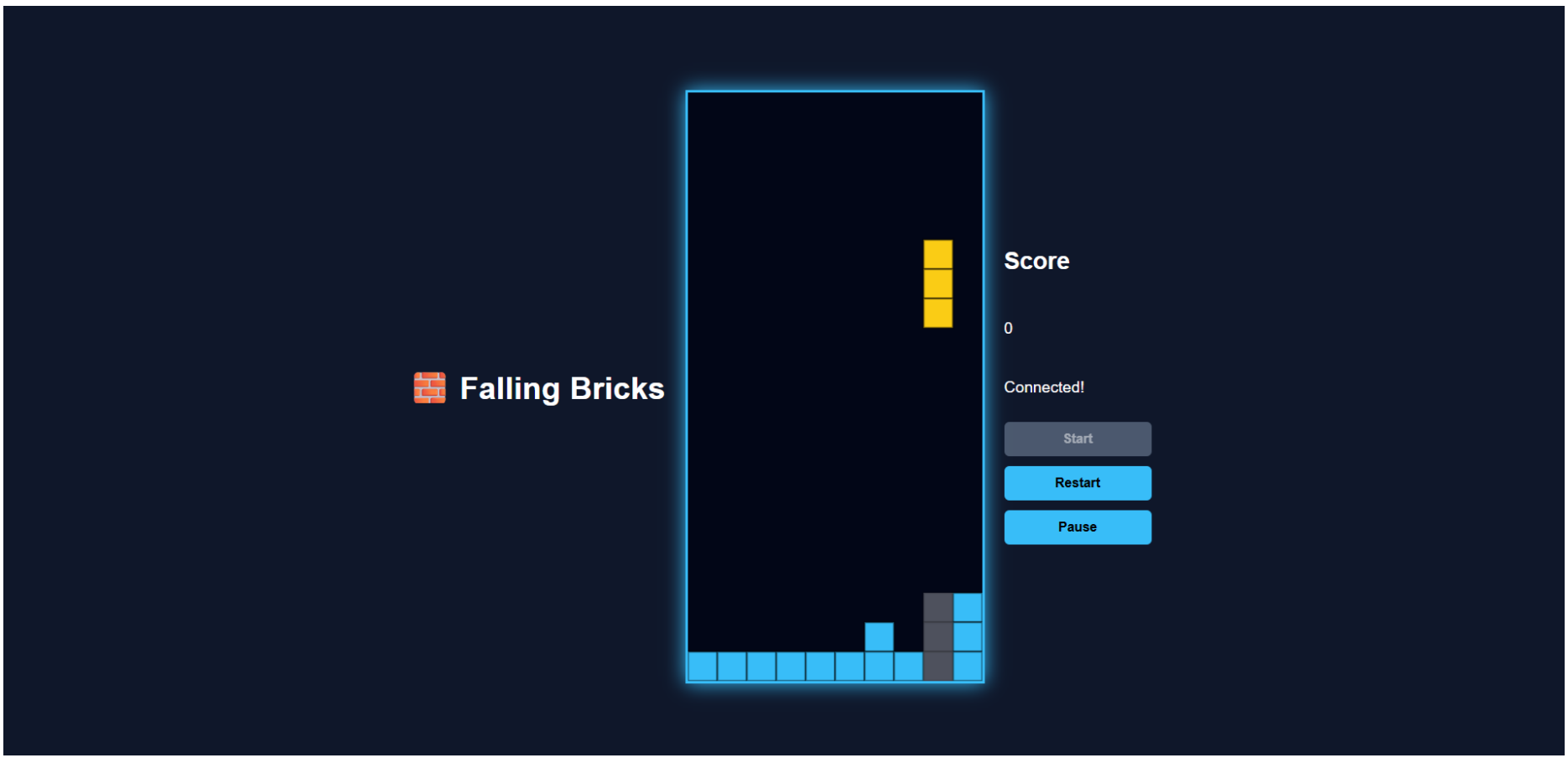
1. Navigate to the project folder (where pom.xml exists)
2. Run: mvn spring-boot:run
3. Open in browser: http://localhost:8080/index.html

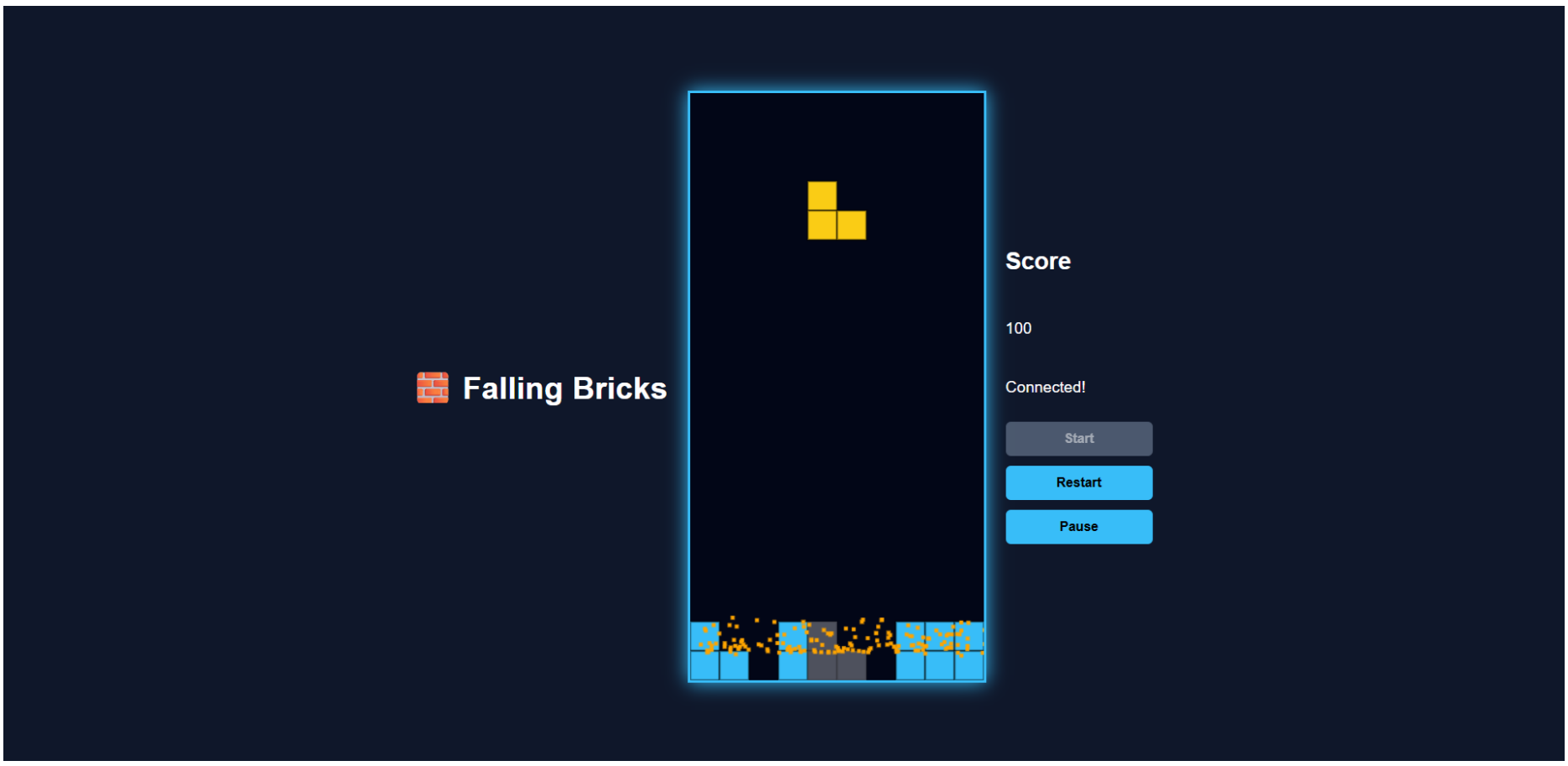
Option 2: Using Prebuilt JAR

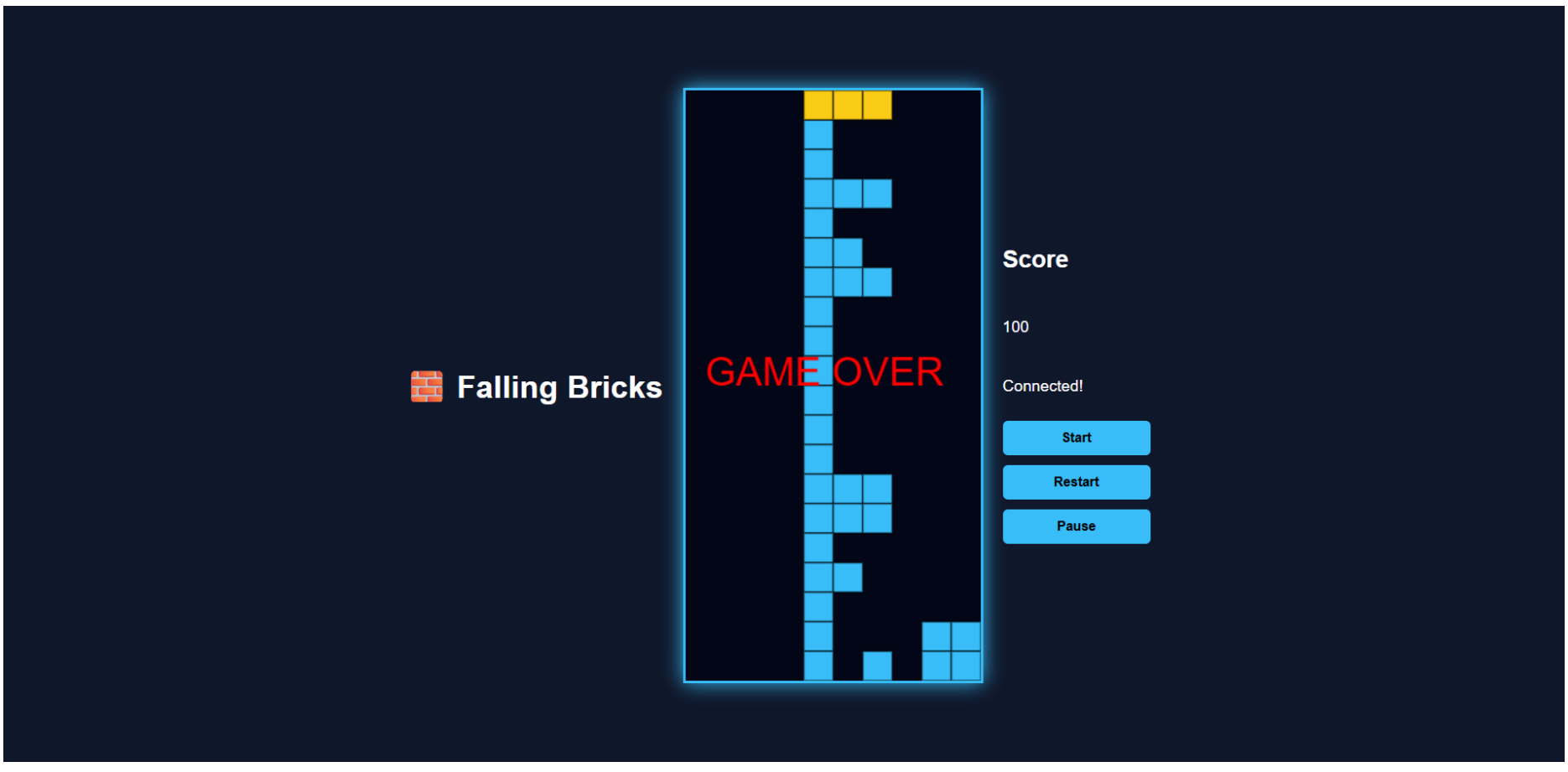
A prebuilt runnable package is available.

1. Navigate to the runartifacts folder
2. Execute: run.bat file
3. Open in browser: http://localhost:8080/index.html

Once application is launched should see screens like below:







📁 Project Structure

⚙️ Backend Requirements

Technologies

- Java 17
- Spring Boot 4.0.5
- WebSocket (STOMP + SockJS)
- Maven
- Lombok

Responsibilities

- Serve frontend assets
- Manage game state
- Handle real-time communication via WebSockets

- Execute continuous game loop with difficulty scaling

🎮 API (WebSocket-based)

Method	Endpoint	Description
POST	/app/start	Start new game
POST	/app/reset	Reset game
POST	/app/pause	Pause/unpause
POST	/app/move	Move brick (LEFT, RIGHT, FAST_DROP, ROTATE)

🔌 WebSocket Flow

- Client connects to /ws-game
- Sends messages to /app/*
- Server broadcasts updates to /topic/game

🧠 Core Components

GameWebSocketController

- Receives /app/* messages
- Calls GameService & GameLoop
- Broadcasts GameState to /topic/game

GameLoop

- Runs scheduled loop
- Handles:
  - Automatic downward movement
  - Difficulty scaling (speed increase)
- Broadcasts updated game state

GameService

Handles all core logic:

- 10x20 grid board
- Brick shapes (I, L, T, etc.)
- Collision detection
- Gravity system
- Line clearing
- Score calculation
- Game over detection
- Wall-kick rotation
- Ghost piece (landing preview)

🎨 Frontend Requirements

Technologies

- HTML5 Canvas
- JavaScript (ES6)
- CSS3
- Web Audio API
- SockJS + STOMP

📄 **index.html**

- Canvas (300x600, 10x20 grid)
- Side panel:
  - Score
  - Status
  - Buttons (Start, Restart, Pause)
- Includes:
  - style.css
  - game.js
  - SockJS & STOMP

Button Behavior

- **Start**
  - Enabled after WebSocket connection
  - Disabled after click
- **Restart**
  - Resets game
  - Re-enables Start
- **Pause**
  - Toggles pause state

🎮 **game.js**

Handles:

WebSocket

- Connects to /ws-game
- Sends:
  - /app/start
  - /app/reset
  - /app/pause
  - /app/move

Controls

- ArrowLeft → Move left
- ArrowRight → Move right
- ArrowDown → Fast drop
- ArrowUp → Rotate
- P → Pause

Rendering

- Board & bricks
- Ghost piece
- Brick borders
- Particle effects (line clear)
- Score display
- Game over message

Sound

- movement.wav plays on:
  - Manual movement
  - Automatic downward movement
- Optional line-clear sound

UI Logic

- Start disables after click
- Restart resets state
- Pause toggles game

🎨 **style.css**

- Canvas layout
- Side panel styling
- Button styles
- Particle visuals

✨ **Features Implemented**

- Real-time WebSocket engine (no polling)
- Start button disable logic
- Restart functionality
- Pause functionality
- Live score updates
- Ghost piece preview
- Wall-kick rotation
- Particle effects (line clear)
- Visible brick borders
- Sound effects (movement.wav)
- Fast drop
- Game over detection
- Difficulty scaling (speed increases)
- Responsive UI

🔄 **Game Workflow**

1. Open browser → WebSocket connects
2. Start button enabled

3. Click Start → game begins
4. Use arrow keys to control bricks
5. Pause/unpause using P or button
6. Score updates live
7. Line clears trigger particle animation
8. Ghost piece shows landing position
9. Restart resets game
10. Game over → Start re-enabled
11. All movements trigger sound

📦 **Deliverables**

- Complete Spring Boot project
- Fully functional frontend & backend
- WebSocket + REST endpoints
- Real-time gameplay
- Sound, particles, ghost piece, rotation
- Ready to run via Maven
- Well-structured codebase
- Complete README documentation

📝 **Notes**

- GameWebSocketController fully handles frontend communication
- GameLoop manages:
  - Auto brick movement
  - Difficulty scaling
  - State broadcasting
- movement.wav is triggered on every brick movement (manual + automatic)